An LLM Wiki for Higher Education: Deterministic Catalog Tools with Selective Hybrid Retrieval for a Bilingual University Advisor

Alex Carreras Forns* · **Josep Carreras Molins** · **Claude Code**

alexcarrerasforns@gmail.com · joe.carr.data@gmail.com

Live demo: https://lasalle.generateeve.com · Source: https://github.com/joe-carr-data/lasalle-wiki-tutor

Abstract

University catalogs at one-institution scale sit in an awkward middle ground for large language model applications. They are too large to fit in any reasonable prompt and too small to justify the operational machinery of a managed vector database. The modern default response to this regime — chunk the documents, embed them, retrieve them at query time — inherits well-documented failure modes around chunking boundaries, retrieval miss, and generation drift [1], and provides no native answer to the long-context degradation that the chunk-then-stuff pattern induces [2], [3]. We present the *LaSalle Wiki Tutor*, an AI study advisor for La Salle Campus Barcelona (Universitat Ramon Llull) that takes a different approach. The catalog is crawled once into a compiled Markdown wiki of 357 program records and 4,606 subject records mirrored across English and Spanish, each with strict YAML frontmatter. A streaming agent built on Agno and OpenAI's Responses API navigates the wiki through ten deterministic, read-only tools rather than through a vector-database substrate. Vector retrieval is preserved as a primitive, but is confined to the inside of one tool: the free-text `search_programs` tool blends BM25-F field-weighted lexical scoring [4] with Model2Vec 256-dimensional static embeddings [5] at a 0.55/0.45 ratio after pool-normalisation, modulated by a small intent prior. On a 20-query benchmark that spans colloquial, bilingual, and rare-token cases, this hybrid configuration is the only mode that reaches 100% top-5 coverage; the lexical-only mode wins top-1 by a five-point margin, and the legacy token-overlap baseline reaches only 45%. The system is observable by construction: three MongoDB collections (Agno sessions, conversation metadata, and per-turn reasoning plus tool-timing traces) supply the empirical material reported here. We run the service publicly at https://lasalle.generateeve.com on a single Terraform-provisioned `t3.micro` behind a shared-token gate, with a measured median per-turn cost of $ 0.013 and a median per-conversation cost of $ 0.006 against a fixed monthly infrastructure cost of approximately $ 14. The implementation is open source at https://github.com/joe-carr-data/lasalle-wiki-tutor.

Keywords— LLM agents; tool use; retrieval-augmented generation; BM25; static embeddings; university advising; observability; system demonstration

* *Corresponding author. This work was prepared in support of the first author's application to LaSalle Campus Barcelona's Bachelor in Artificial Intelligence and Data Science.*

1 Introduction

Consider a prospective student typing into a chat interface in Spanish: *¿qué diferencia hay entre el grado en Inteligencia Artificial y el de Ingeniería Informática?* – what is the difference between the AI bachelor and the Computer Engineering bachelor? The answer exists. It is split across two program pages on a university website, each with frontmatter that fixes the exact number of ECTS credits, the modality, the start date, the language of instruction, and a year-by-year curriculum. The institution publishes this information as the source of truth for student decisions. The student would like a direct answer, with citations to those pages, in Spanish, in under twenty seconds.

This is the regime the LaSalle Wiki Tutor is built for. The institution it advises is La Salle Campus Barcelona, a faculty of Universitat Ramon Llull, with a bilingual academic catalog of 357 program records and 4,606 subject records spread across an English and a Spanish web tree. The scale matters: the catalog is too large to fit into any reasonable prompt, and too small to repay the

operational machinery of a managed vector database with its index lifecycle, embedding drift, and serving overhead. It is also too high-stakes for the failure modes that come with naive retrieval-augmented generation: a wrong ECTS count, a hallucinated faculty name, or a fabricated tuition figure damages the institution.

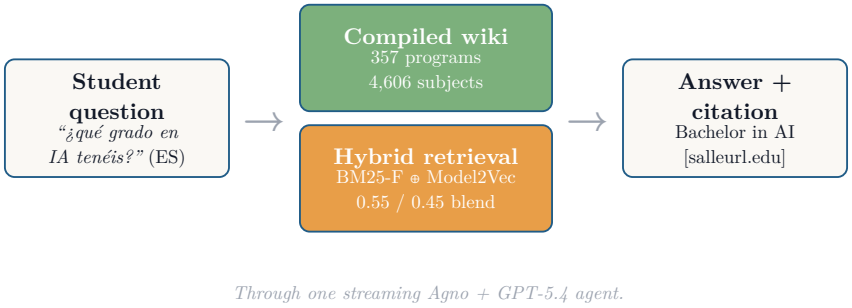


Figure 1. Graphical abstract. The system compiles the catalog once into a structured wiki, exposes ten deterministic read-only tools to a streaming agent, and confines vector retrieval to one tool. Answers cite the institution's own pages.

Figure 1 surveys the architecture this paper defends. Three design pressures shape what follows. First, *bilingual symmetry*: every program may exist in English, in Spanish, in both, or in neither, and the system must answer fluently in whichever language the student wrote, with citations to the corresponding language's source page. Second, *mid-scale knowledge*: the catalog is bounded, structured, and slowly changing; it does not call for the affordances of a vector database, but it does call for something more disciplined than putting the whole thing into a system prompt. Third, *institutional cost of being wrong*: every claim must be grounded in the catalog or it must not be made at all.

The thesis of this paper is that for knowledge bases of this shape – bounded, structured, slowly changing – a curated and compiled Markdown wiki navigated by a small set of deterministic tools is a more honest substrate than the default chunk-and-embed retrieval pipeline. This argument is recognisably the *LLM Wiki* pattern that [6] articulated as a design proposal, grounded in the broader framing of large language models as an emerging operating system whose context window functions as scarce RAM [7]. It is also empirically defensible against the failure modes of naive retrieval-augmented generation [1] and against the long-context degradation that the alternative of "stuff everything into the prompt" induces in even very capable models [2], [3].

We do not reject vector retrieval as a primitive. We reject it as the agent's general substrate. Inside one tool – the free-text `search_programs` – the system runs a hybrid blend of BM25-F field-weighted lexical scoring [4] and Model2Vec 256-dimensional static embeddings [5], in the lineage of dense-retrieval methods [8], [9], modulated by a small intent prior. Section 3 documents the precise blend. The remaining nine tools are deterministic file reads with structured arguments and structured returns. The agent navigates this surface using the tool-use lineage of [10] and, in spirit, the selective-retrieval refinement of [11], in which the model chooses *when* to consult an external source rather than always doing so.

This paper makes four contributions. First, we document a field-targeted extraction pipeline that produces structured Markdown with strict YAML frontmatter from a Drupal-based university catalog, with a measured fallback rate of 1.6%. Second, we present a ten-tool surface that the system prompt bounds to at most five tool calls per response; the production trace store shows that catalog-shaped student questions typically resolve well below this ceiling (Section 5.3). Third, we present a hybrid retrieval design — BM25-F + Model2Vec — confined to a single tool, and we ablate it across four ranker modes on a 20-query benchmark. Fourth, we describe an observability-first runtime in which three MongoDB collections record reasoning passages, tool timings, and conversation metadata, supplying the empirical material this paper reports. The system runs

publicly at https://lasalle.generateeve.com the deployment recipe, access mechanism, and code-and-data availability statement are in Section 4.

2 Related Work

2.1 Retrieval-augmented generation and its failure modes

The canonical formulation of retrieval-augmented generation pairs a dense retriever over a document collection with a generator conditioned on the retrieved passages [12], building on dense passage retrieval [9] and bi-encoder embedding methods [8]. The pattern is now the default reflex when an LLM application needs to ground its answers in external text. It is also empirically fragile. [1] catalogues seven distinct failure modes in production RAG systems – missing content, retrieval at the wrong granularity, top-K cutoffs, chunking-boundary information loss, generation drift, formatting drift, and version drift between the index and the source – each of which is invisible at small evaluation scale and recurs in deployment. The LaSalle Wiki Tutor avoids the chunking-boundary and granularity classes by construction: there is no chunking. The retrieval surface is the wiki's natural unit (a program record, a subject record, a curriculum section), and the agent calls a deterministic tool to return it. The retrieval miss class is mitigated by the hybrid lexical-and-semantic blend inside `search_programs` (Section 3.5), the wider tool surface that lets the agent recover via `list_programs` or `get_index_facets` when free-text retrieval underperforms, and the structured frontmatter that gives the BM25-F scorer rich short-text fields to weigh.

[11] occupies an interesting position in this lineage: a refinement of RAG in which the model emits reflection tokens that decide when to retrieve, what to retrieve, and how to critique the retrieved evidence. The LaSalle Wiki Tutor is not Self-RAG, but it shares Self-RAG's premise that retrieval should be on-demand rather than always-on. The mechanism is different: where Self-RAG decides whether to retrieve at the token level, our system decides whether to retrieve at the tool-call level, and which structured tool to call. The point of contact is the recognition that always retrieving is not the right default.

2.2 Long-context degradation and conversation drift

A naive alternative to RAG for a corpus of our size would be to inline the whole wiki into the system prompt. [2] measures the empirical cost of this strategy: even very capable language models lose accuracy when relevant facts sit in the middle of a long context, with a U-shaped curve that favours the beginning and the end of the window. The effect compounds in multi-turn conversation: [3] reports a roughly 39% performance drop in multi-turn settings versus single-turn baselines, attributed to a "loss of middle turns" in which the model effectively forgets information that arrived in earlier exchanges. Both findings argue against the conversation history being the primary substrate for catalog facts. The LaSalle Wiki Tutor responds to this evidence in two ways. We bound conversation history to the eight most recent runs via Agno's `num_history_runs=8`, and we have the agent re-fetch through tools on every turn rather than relying on the model's recall of earlier tool returns. The cost is a small extra tool call per turn; the benefit is that the answer is grounded in current frontmatter rather than in a possibly-stale recollection.

2.3 Foundational retrieval and the case for BM25-F

The case for keeping a lexical scorer in the retrieval pipeline rests on a literature that predates large language models. Term-frequency-inverse-document-frequency weighting was formalised by [13] and remains the analytical foundation of BM25, whose probabilistic-relevance derivation and field-weighted extension to BM25-F are documented authoritatively in [4]. For a corpus whose primary units are short and frontmatter-heavy – a program record's title, tags, area, and level field together account for most of the discriminative signal – a field-weighted lexical scorer is hard to beat with dense embeddings alone, and the ablation in Section 5.2 is consistent with this prior: the lexical-only mode wins top-1 by five points and ties hybrid at top-3. The role of the dense signal is at the tail: at top-5, hybrid reaches 100% only because the semantic branch catches the queries where literal-token overlap is zero.

2.4 Static embeddings and the agent-tools lineage

The semantic pole in our retrieval pipeline is intentionally lightweight. Model2Vec [5] distils static word embeddings from a transformer-based sentence encoder, producing fixed-size per-token vectors that can be pooled into a sentence-level representation without a transformer pass at query time. The peer-reviewed counterpart to this approach is the static-word-embedding-for-sentence-representation line of work [14], which characterises the design space (PCA on Sentence Transformer outputs, contrastive distillation) and shows that the resulting static embeddings approach the recall quality of their teachers at a fraction of the runtime cost. For our scale (357 programs, embeddings precomputed once at wiki-render time) the cost-quality balance lands cleanly: a 256-dimensional vector per program, a 180 kB `.npz` file per language, a numpy matrix-vector multiply per query.

The agent-tools half of the architecture is recognisable as the ReAct lineage [10], in which a language model interleaves reasoning steps and tool actions. The LaSalle Wiki Tutor is closer to a constrained ReAct than to the full Self-RAG-style critic loop: the model emits reasoning, picks a tool, reads the JSON return, and continues. The constraints are the system prompt's "at most five tool calls" budget and the read-only nature of the tool surface. There is no write tool, no critic loop, no plan-and-execute scaffolding.

2.5 The design philosophy: the LLM Wiki

The non-peer-reviewed design proposal that most directly captures the architectural commitment we make is [6], which articulates the "LLM Wiki" pattern as a three-layer design: raw immutable sources, an LLM-curated wiki layer that synthesises them, and a schema document (the gist proposes `CLAUDE.md` or `AGENTS.md`) that governs structure and conventions. The design philosophy frames the wiki as a compiled, persistent artefact rather than a query-time synthesis, and it sits within [7]'s broader "Software 3.0" framing in which LLMs are an emerging operating system with the context window functioning as scarce RAM. Both sources are non-peer-reviewed and we cite them as primary sources for the design philosophy under defence, not as evidence for our empirical claims; the evidential weight in this paper rests on the peer-reviewed work surveyed above.

The LaSalle Wiki Tutor differs from the LLM Wiki proposal in one important respect: the wiki in our system is built by a deterministic extraction pipeline from the institution's published catalog, not maintained by the LLM itself. The institution is the source of truth; the agent is read-only. This is the right specialisation for a high-stakes, single-institution advisor: the corpus is already curated, and the question is how to navigate it cleanly, not how to maintain it.

3 System Description

3.1 Overview and design commitments

The LaSalle Wiki Tutor is structured as five sequential stages and one observability spine. The catalog is crawled into raw HTML, the raw HTML is rendered into a Markdown wiki with strict frontmatter, the wiki is wrapped by a read-only Python API that exposes ten tools, the tools are bound to a streaming agent that runs over Server-Sent Events, and the SSE stream feeds a React client served from the same FastAPI process. Three MongoDB collections record what happened. Figure 2 shows the shape.

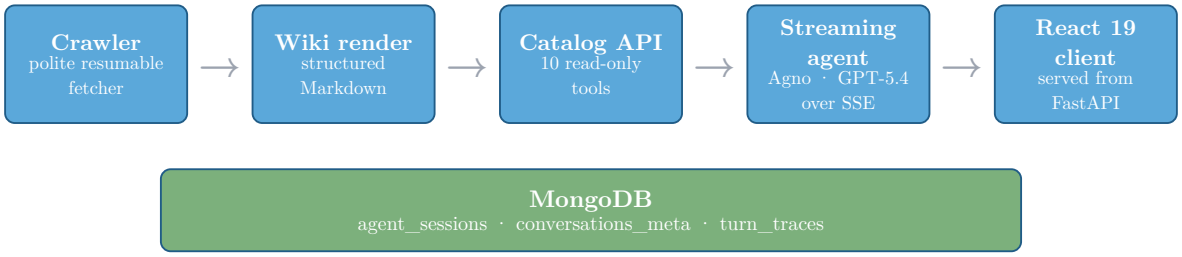


Figure 2. System architecture. Five stages from crawl to client; one MongoDB store for observability and conversation history.

Four design commitments anchor the rest of this section. We extract structured records by targeting Drupal field selectors rather than running whole-page HTML-to-Markdown conversion. We bound the agent's view of the catalog to ten deterministic tools, with a system-prompt-enforced budget of at most five tool calls per response. We confine vector retrieval to one tool. We record per-turn reasoning passages and tool timings on the side, never on the SSE hot path, so the evaluation in Section 5 has measured ground truth.

3.2 Crawl and corpus construction

The source corpus is the bilingual academic catalog at `salleurl.edu`, a Drupal-based site that publishes one URL per program in each language plus a hierarchy of per-program subpages (goals, requirements, curriculum, methodology, faculty, careers). The site does not publish a sitemap; the published `robots.txt` requests a 10-second crawl delay. The crawler in `scripts/fetch_catalog.py` walks the program index in three phases (enumerate seeds, fetch programs and subjects, verify integrity), is resumable across interruption, and honours the requested delay. We measure 179 English program records and 178 Spanish program records (357 total programs), with 2,314 English and 2,292 Spanish subject records (4,606 subjects in total); these counts are computed directly from `wiki/meta/catalog.jsonl` and `wiki/meta/subjects.jsonl` rather than from the build-summary file `wiki/meta/stats.md`, which is a downstream artefact. Figure 3 traces the three crawl phases and the field-targeted extraction step that hands off to the pairing stage described below.

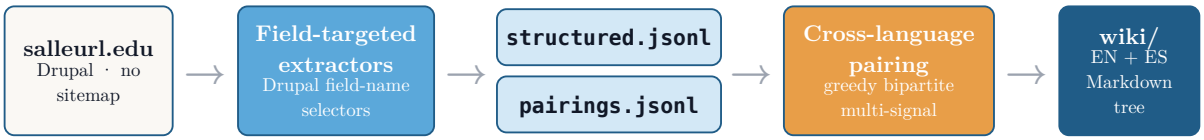


Figure 3. The crawl-and-build pipeline. Field-targeted extraction produces structured JSONL; cross-language pairing runs greedy bipartite matching over four signals.

Cross-language pairing assigns each English program record an `equivalent_program_id` pointing to its Spanish counterpart (and vice versa) where one exists. The matcher in `scripts/build_wiki.py` scores every EN–ES pair on four signals (slug similarity, title similarity, shared subject URLs, and a structural-attribute match over ECTS, duration, and modality), combined with weights 0.20, 0.20, 0.45, and 0.15 respectively. A pair is auto-linked when the weighted score exceeds 0.30, OR when shared subjects and structural match both clear 0.5, OR when title and slug similarity jointly clear permissive thresholds, OR when title and structural agreement are both very high. Matching is greedy bipartite: each Spanish program can be claimed only once. We measure 103 auto-linked pairs out of 179 English programs, a link-coverage rate of 57.5%. The remaining 76 unlinked English programs cluster heavily in short-format levels, computed against the English-side counts as the denominator: 28 of 49 specialisations (57%), 27 of 36 "other" entries (75%), and 8 of 25 online courses (32%) remain unlinked. By contrast, only 1 of 29 English bachelors and 11 of 38 English masters are unlinked, so the major degree types are captured well and the unlinked tail corresponds largely to short courses where genuine single-language existence is common. The `pairing_confidence` field stored in the frontmatter is the weighted-score signal, not a probability

of correctness; the OR-rules deliberately allow correct pairs with low scores through. Section 5.1 reports the rule-fired decomposition, which shows that 45% of auto-linked pairs sit in the 0.30–0.50 weighted-score band and would have been missed by a single-threshold approach.

3.3 Field-targeted extraction and the wiki schema

The wiki render step takes the raw HTML and produces structured records, not Markdown blobs. For each program subpage the extractor looks for Drupal field classes (`field-name-overview`, `field-name-careers`, and so on) and runs `markdownify` only on the inner HTML of each named field. This avoids the noise that whole-page HTML-to-Markdown produces – navigation chrome, breadcrumbs, footer boilerplate, and cookie banners – which would otherwise inflate field sizes and pollute the downstream retrieval body. A field that fails to extract triggers a tracked fallback path so corpus health is monitorable. We measure a fallback rate of 1.6%, comfortably below the 5% target we set for ourselves.

```
wiki/
├── INDEX.md, faq.md, glossary.md
├── meta/
│   ├── catalog.jsonl        (357 program records)
│   ├── subjects.jsonl       (4,606 subject records)
│   ├── pairings.jsonl       (EN↔ES candidates)
│   ├── embeddings_en.npz    (Model2Vec 256-d)
│   └── embeddings_es.npz
├── en/
│   ├── INDEX.md, by-area/, by-level/
│   ├── programs/{slug}/
│   │   ├── README.md         (frontmatter + overview)
│   │   ├── goals.md, requirements.md
│   │   ├── curriculum.md, careers.md
│   │   └── methodology.md, faculty.md
│   └── subjects/{slug}.md
└── es/     (mirrors en/, /estudios/ slug)
```

Program frontmatter schema

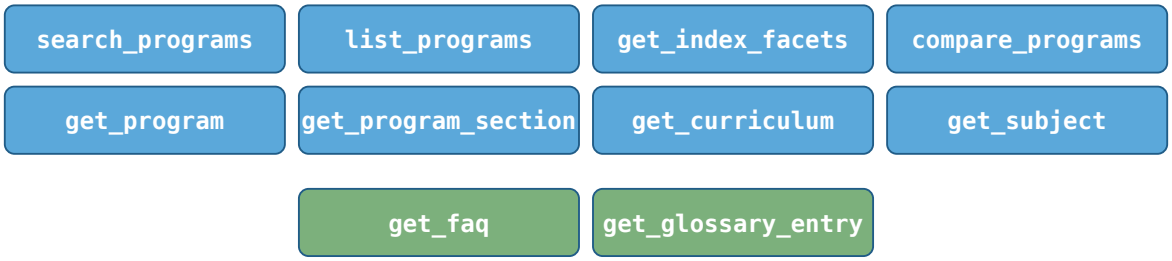
- `title`, `slug`, `canonical_program_id`
- `level`, `area`, `official`, `tags`
- `modality` (array), `duration`, `ects`
- `languages_of_instruction` (array)
- `schedule`, `location`, `start_date`
- `tuition_status`, `admissions_contact`
- `official_name`, `degree_issuer`
- `subject_count`, `related_programs`
- `equivalent_program_id`
- `pairing_confidence`, `pairing_method`
- `source_url`, `source_fetched_at`
- `extractor_version`, `extractor_mode`
- `last_built_at`

Figure 4. On-disk wiki layout and program frontmatter schema. Each program is a directory of Markdown files with YAML frontmatter; each subject is a single Markdown file. Sidecars in `meta/` carry the full frontmatter dump, EN–ES pairings, and the static-embedding `.npz` matrices.

The wiki layout mirrors the catalog's natural hierarchy. Figure 4 lays out the directory tree alongside the program frontmatter schema. Frontmatter fields are populated at the following measured rates: eighteen fields (`title`, `slug`, `canonical_program_id`, `level`, `area`, `modality`, `duration`, `languages_of_instruction`, `schedule`, `location`, `start_date`, `tuition_status`, `admissions_contact`, `official_name`, `source_url`, `source_fetched_at`, `extractor_version`, and `official`) are present in 100% of records; `ects` is present in 95.5%, `subject_count` in 86.6%, `tags` in 59.4%, `equivalent_program_id` and `pairing_confidence` in 57.7% (the same population as the auto-linked pairs by construction), and `degree_issuer` in 49.9%. The completeness ratios that matter for retrieval (the lexical-search fields and the language-and-level facets) are at or above 86%.

3.4 The ten-tool surface

The agent reaches the wiki through ten tools, each a Python coroutine wrapping a function in `catalog_wiki_api`. The tools fall into three families. The browsing family covers the cases where the student is exploring: `search_programs` for free-text queries, `list_programs` for filtered structured browse, `get_index_facets` for the area-and-level inventory, `compare_programs` for side-by-side comparisons. The detail family covers focused look-ups: `get_program`, `get_program_section`, `get_curriculum`, `get_subject`. The routing family covers meta questions where the catalog itself contains the answer: `get_faq` and `get_glossary_entry`. The system prompt tells the agent to spend at most five tool calls per response and to prefer the cheapest tool that resolves the student's question; the tools themselves return structured JSON with explicit `ok` and `error` fields so the model can recover from a missing argument or a not-found id without escalating. Figure 5 lays out the ten tools by family.



Blue: retrieval and detail tools. Green: routing tools (FAQ, glossary).

Figure 5. The ten-tool surface, grouped by family. Blue tiles are retrieval and detail tools; green tiles are the routing tools that answer meta questions directly from the catalog. Two example call sequences the agent typically takes: an explorer follows `get_index_facets` → `list_programs` → `get_program`, and a comparison shopper follows `search_programs` → `compare_programs` → `get_curriculum`.

Every tool wraps its public API call in an argument-coercion layer that the agent's strict-mode tool-calling occasionally needs. Optional string filters are normalised so the literal empty string becomes a missing filter, and required string arguments that arrive empty return a structured `missing_argument` error with a hint telling the model which sibling tool to call to obtain a usable value. This pattern – normalise where the model is permissive, return a structured hint where the model is wrong – keeps the tool surface forgiving without softening the API contract.

3.5 Hybrid retrieval inside `search_programs`

`search_programs` is the only tool in which vector retrieval lives. Its job is to take a free-text query and return a ranked list of programs from one of the language catalogs. The retrieval pipeline computes two scoring branches that are then blended: a lexical branch and a semantic branch. Each branch is pool-normalised into the closed interval [0, 1] over the candidate pool, the two are combined linearly with weights 0.55 and 0.45, and a small intent prior is multiplied in last that nudges substantive degree-level programs up and short-format courses down when the query carries cues like "bachelor", "master", or "doctorate" and their Spanish equivalents. Figure 6 traces the full data flow from query to ranked top-k.

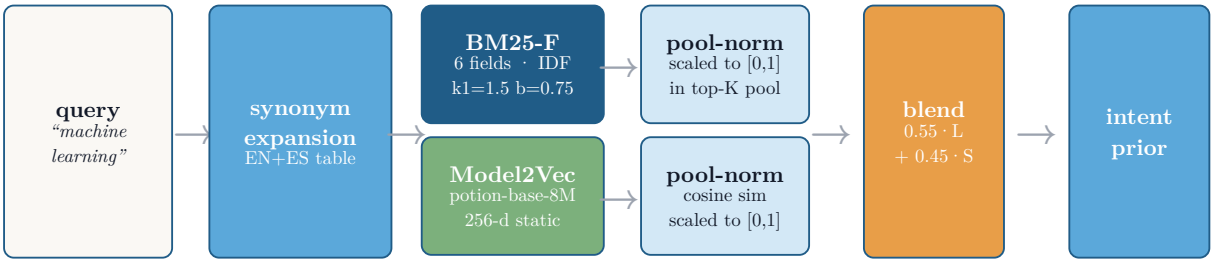


Figure 6. Hybrid retrieval inside `search_programs`. BM25-F is the lexical pole; Model2Vec is the semantic pole; both branches are pool-normalised before the linear blend and a small intent prior is multiplied in last.

The lexical pole is BM25-F with six fields (title 4.0, tags 3.0, area 2.0, level 1.5, body 1.0, slug 0.5), a Robertson–Spärck-Jones IDF [13], and the standard BM25 hyperparameters $k_1 = 1.5$ and $b = 0.75$ [4]. Queries pass through a synonym-expansion table before scoring (so "hacking" expands to include "cybersecurity"; "aprendizaje automático" expands to "machine learning"). The semantic pole uses Model2Vec's `minishlab/potion-base-8M` model [5], a static distilled embedding family that produces fixed-size 256-dimensional vectors per program without a transformer encoder pass at query time, in the family characterised by recent peer-reviewed work on static sentence representations [14]. The embeddings are pre-built once at wiki-render time and stored as L2-normalised `numpy` arrays in `wiki/meta/embeddings_en.npz` and `wiki/meta/embeddings_es.npz`; query embedding takes a few milliseconds, cosine similarity is a single matrix-vector multiply, and the whole pipeline serves from a `t3.micro` with no specialised infrastructure.

Both signal streams are normalised within the candidate pool before blending, not against absolute corpus statistics. This is the right choice for a 357-program catalog: a query that produces a single near-perfect lexical hit and three weak ones should not have its lexical score crushed against the semantic distribution, and vice versa. The blend weights 0.55 lexical and 0.45 semantic were chosen empirically against a development benchmark; the value of the lexical bias for a front-matter-heavy catalog is visible in the ablation reported in Section 5.2. The `LASALLE_RANKER_MODE` environment variable selects among `hybrid`, `lexical` (BM25-only), `semantic` (cosine-only), and `token_overlap` (the legacy baseline retained for regression comparison). The same environment variable feeds the four-mode ablation reported below.

3.6 Streaming agent runtime and observability

The agent itself is a thin wrapper over Agno's `Agent` class, configured to use OpenAI's Responses API with `gpt-5.4` at medium reasoning effort. The agent maintains eight most-recent runs of conversation history via Agno's `num_history_runs=8`, with MongoDB session persistence backed by the `wiki_tutor_agent_sessions` collection. Token budgets and tool-call budgets are enforced at the prompt level rather than via API parameters: the system prompt explicitly tells the model to stop at five tool calls per turn unless an honest answer truly requires more, and the empirical median over the production trace store is well below this limit (Section 5.3).

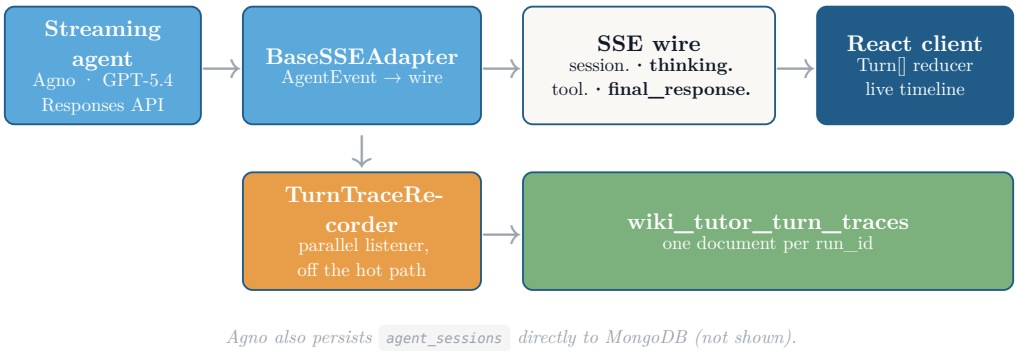


Figure 7. SSE event lifecycle and observability fan-out. The `BaseSSEAdapter` is in the hot path; the `TurnTraceRecorder` is a parallel listener that writes one document per `run_id` to the trace collection.

Figure 7 shows the streaming hot path and the parallel observability listener. Events flow over Server-Sent Events between the agent and the client. The vocabulary covers session lifecycle (`session.started`, `session.ended`), reasoning (`agent.thinking.start`, `agent.thinking.delta`, `agent.thinking.end`), tool execution (`tool.start`, `tool.end`), and the final response (`final_response.delta`, `final_response.end`, `response.final`). The wire format wraps each event in an envelope that carries the event name, timestamp, elapsed milliseconds, correlation id, and agent metadata; the payload sits under a `data` field. The client's React reducer is structured so that delta events update only the rendered text; non-delta events transition the turn through a small state machine. The system supports cooperative cancellation through a process-local registry that pairs each in-flight `query_id` with an `asyncio.Event` the agent loop polls between tool calls.

Observability is the spine that makes the rest of the paper measurable. Three MongoDB collections cooperate without coupling. The `wiki_tutor_agent_sessions` collection is Agno's own session record, which we treat as read-only and join against on demand. The `wiki_tutor_conversations_meta` collection is the application's title-and-language sidecar, with a `version` field that gives the rename endpoint optimistic-concurrency semantics and a `deleted_at` field that supports soft delete. The `wiki_tutor_turn_traces` collection is the empirical workhorse: one document per `run_id`, populated by a `TurnTraceRecorder` that listens on the same event manager as the SSE adapter but writes only on turn completion, so it never sits on the streaming hot path. Each trace document carries every reasoning passage with timestamps, every tool invocation with arguments preview and duration, the language, the user id, the start and end of

the turn. This is what we read in Section 5.3 to report per-tool latency without instrumenting the production code.

The agent's system prompt also enforces a scope boundary. The agent is instructed to answer in-scope questions about the LaSalle catalog (program details, comparisons, curricula, careers, FAQ-style routing) and to decline politely, in the user's language, when the question is out of scope (other universities, general knowledge, coding help, personal or visa advice, writing on the student's behalf). The prompt explicitly forbids fabrication when the catalog is silent; tuition is the canonical example, with the agent instructed to route the student to the admissions URL rather than invent a price. The prompt also instructs the model to refuse role-swap and system-prompt-disclosure attempts. Section 5.5 reports a small spot-check of these guardrails on the live deployment.

4 Deployment and Availability

Reviewer access. The deployed system is reachable directly with the credentials below.

Endpoint: https://lasalle.generateeve.com
Source: https://github.com/joe-carr-data/lasalle-wiki-tutor
Access token: *distributed out of band on request to the corresponding author*

Paste the token into the gate UI on first load, or send it as the `X-Access-Token` header on any `/api/wiki-tutor/*` request. The token is rotated independently of this release; if it has been rotated, contact the corresponding author for the current value.

The system runs publicly at https://lasalle.generateeve.com. The deployment is intentionally minimal so the cost story is honest: a single AWS EC2 `t3.micro` in the `eu-west-1` region, with Caddy terminating TLS via Let's Encrypt on ports 80 and 443, `uvicorn` serving FastAPI under `systemd` on the loopback interface, and `mongo:6.0` running in `docker compose` on the same host. The fixed monthly cost is approximately fourteen US dollars (`t3.micro` at $7.50, a 30 GB `gp3` root volume at $2.40, a public IPv4 at $3.60 since the 2024 pricing change, daily EBS snapshots under $0.50, no Elastic IP charge while attached to a running instance, no managed database charge). Figure 8 shows the topology.

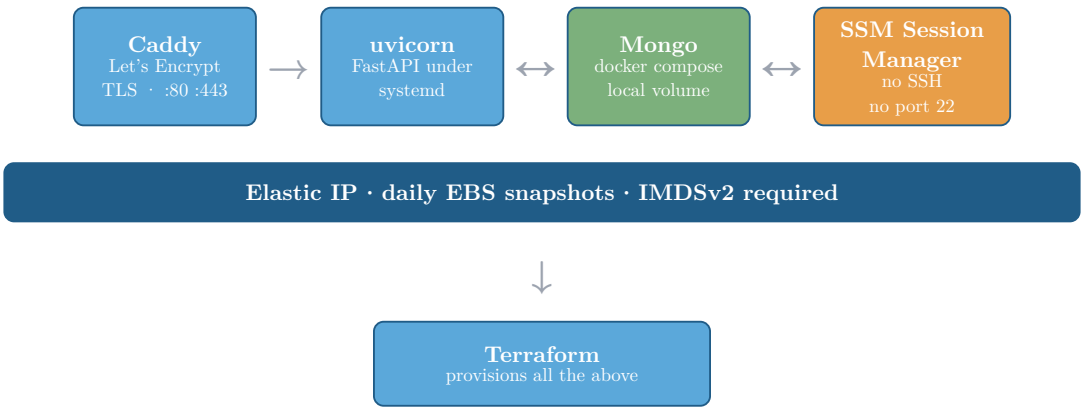


Figure 8. Deployment topology. Single `t3.micro` instance with Caddy, uvicorn, and Mongo in docker compose. SSM Session Manager replaces SSH. Terraform provisions the entire stack.

4.1 Access mechanism

Public access is gated by a single shared-secret token validated server-side. The client presents the token in an `X-Access-Token` header; every `/api/wiki-tutor/*` endpoint applies the `require_access_token` FastAPI dependency, which performs a constant-time string comparison against the `WIKI_TUTOR_ACCESS_TOKEN` environment variable. A separate `/api/auth/validate` endpoint exists for the gate UI to test the token before storing it in the browser; this endpoint is protected by a per-IP rate limiter (ten attempts per minute, with a `Retry-After` header on the

response) so brute-force attempts are bounded. The token in use at the time of writing is reproduced in the reviewer-access box at the head of this section; it is rotated independently of the source release, and the corresponding author distributes the current value out of band on request after rotation.

4.2 Operational discipline

Operations on the instance happen exclusively through AWS Systems Manager Session Manager. There is no port 22 in the security group and no SSH key pair is provisioned. The instance's IAM role carries the `AmazonSSMManagedInstanceCore` managed policy and an inline policy scoped to read the two SSM Parameter Store entries that hold the OpenAI API key and the access token. IMDSv2 is required (`http_tokens = required` , `http_put_response_hop_limit = 1`) so container workloads cannot reach the instance metadata service even if the docker daemon were compromised. The root EBS volume is encrypted at rest. A Data Lifecycle Manager policy snapshots the root volume daily at 03:00 UTC and retains seven snapshots; the snapshots persist after `terraform destroy` and must be manually deleted to stop incurring storage charges.

Updates take two forms. To ship new code we run `aws ssm send-command` with a small inline script that `git fetch && git reset --hard origin/main` , restarts the `wiki-tutor` systemd service, and confirms `/health` returns 200. To rotate the wiki corpus we ship a new GitHub Release tagged `wiki-latest` whose asset is a tarball of the wiki tree, then run the same SSM command pattern to call `scripts/fetch_wiki.sh --force` and restart. The corpus and the application have separate release cadences, which lets the corpus be re-extracted and re-paired without a code change and vice versa.

4.3 Code and data availability

The implementation is open source under the MIT license at https://github.com/joe-carr-data/lasalle-wiki-tutor. The repository contains the crawler, the wiki builder, the catalog API, the agent and tool definitions, the FastAPI server, the React client, the Terraform infrastructure module, and the evaluation scripts that produced every figure and table in this paper. The rendered wiki corpus (approximately 50 MB compressed) is published as a GitHub Release asset under the rolling `wiki-latest` tag and is fetched idempotently by `scripts/fetch_wiki.sh` . The raw HTML scrape inputs (`data/`) are not redistributed; reproducing them from scratch requires re-running the polite crawl against `salleurl.edu` over approximately eight to ten hours. The `paper/` directory of the repository contains the Typst source of this manuscript, the scripts in `paper/scripts/` that produce every data file in `paper/data/` , and the figure-generation code under `paper/scripts/make_figures.py` and `paper/scripts/make_schematics.py` . The diagrams in this paper that render as inline Typst figures are defined in `paper/diagrams.typ` .

5 Evaluation

We report five measurements. Each is reproducible from the scripts in `paper/scripts/` , with raw data dumps in `paper/data/` . Numbers are computed directly from primary sources – catalog JSONL files, MongoDB collections, the live SSE endpoint – and not from build-summary documents.

5.1 Corpus coverage and frontmatter completeness

The corpus consists of 357 programs (179 EN, 178 ES) and 4,606 subjects (2,314 EN, 2,292 ES), counted from `wiki/meta/catalog.jsonl` and `wiki/meta/subjects.jsonl` . Figure 9 shows the area distribution of the English catalog (panel a) and the frontmatter fields with the lowest completeness (panel b). The area distribution is heavy at Business & Management, Architecture, and AI & Data Science, with a long tail through Cybersecurity and Health Engineering. Eighteen frontmatter fields are present in 100% of records; ECTS clears 95% and `subject_count` 86%; the lowest-completeness fields are `degree_issuer` at 49.9% and the pairing fields at 57.7% (which is, by construction, the same population as the auto-linked pairs). All retrieval-critical fields used by the BM25-F scorer are at 100%.

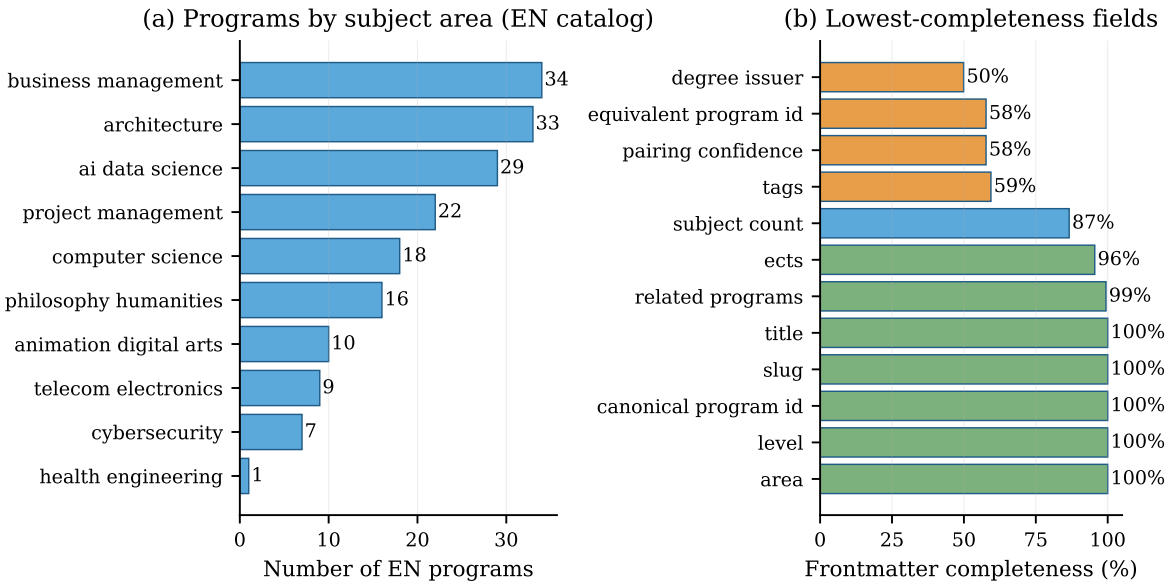


Figure 9. Corpus coverage. (a) Programs by area in the EN catalog. (b) Frontmatter fields ranked by completeness, lowest first. Green is ≥ 95%, blue is 70–95%, orange below 70%.

The pairing decomposition is the most informative coverage diagnostic. Of the 103 auto-linked EN–ES pairs, 36% were captured by the shared-subjects-plus-structural rule, 33% by the title-plus-slug rule, 22% by the weighted-score threshold, and 9% by the title-plus-structural rule (Figure 10, panel a). Panel b breaks the same 103 pairs into score bands: 27 are high-confidence (≥ 0.70), 30 mid (0.50–0.70), and 46 low (0.30–0.50). The low-confidence band is the reason the OR-rules exist. Almost half of the auto-linked pairs are correctly identified by an OR-rule despite their weighted score being below 0.50; a single-threshold matcher with the same 0.5 cut would miss them.

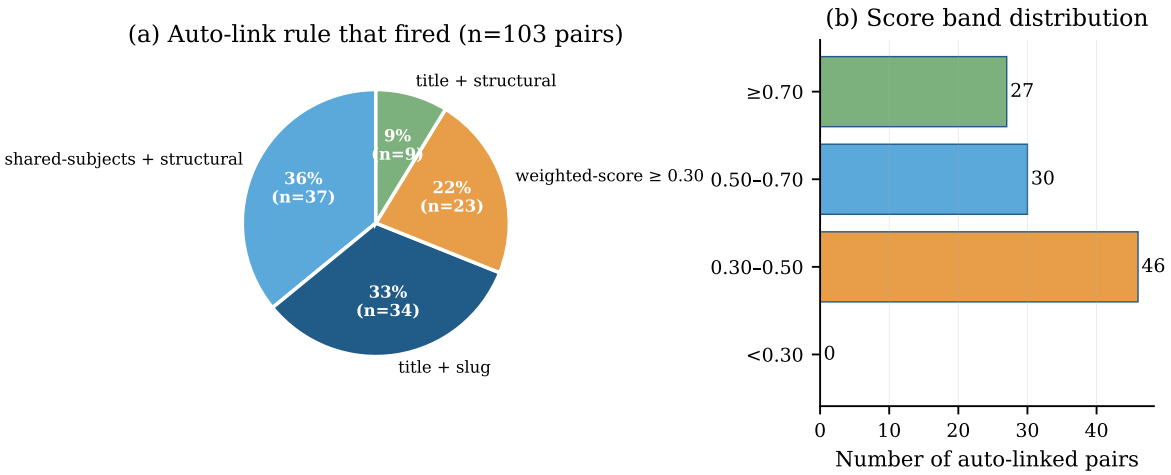


Figure 10. Pairing decomposition. (a) Which OR-rule caught each auto-linked pair. (b) Weighted-score band among auto-linked pairs; 45% sit below 0.50 and would have been missed by a single-threshold matcher.

Figure 11 breaks the linked-versus-unlinked count by level, computed against the English-side counts as the denominator. The bachelors line is essentially complete (1 of 29 English bachelors unlinked); the masters tail has 11 of 38 English masters unlinked; the bulk of the unlinked population sits in specialisations (28 of 49) and the "other" bucket (27 of 36), which matches the institutional reality that short-format and single-discipline offerings are commonly published in one language only. We do not claim that the matcher is correct on the unlinked programs in the sense that they would benefit from a manual review; we claim that the unlinked population

is concentrated in the corpus segments where unpairedness is plausible by inspection of the institution's published catalog.

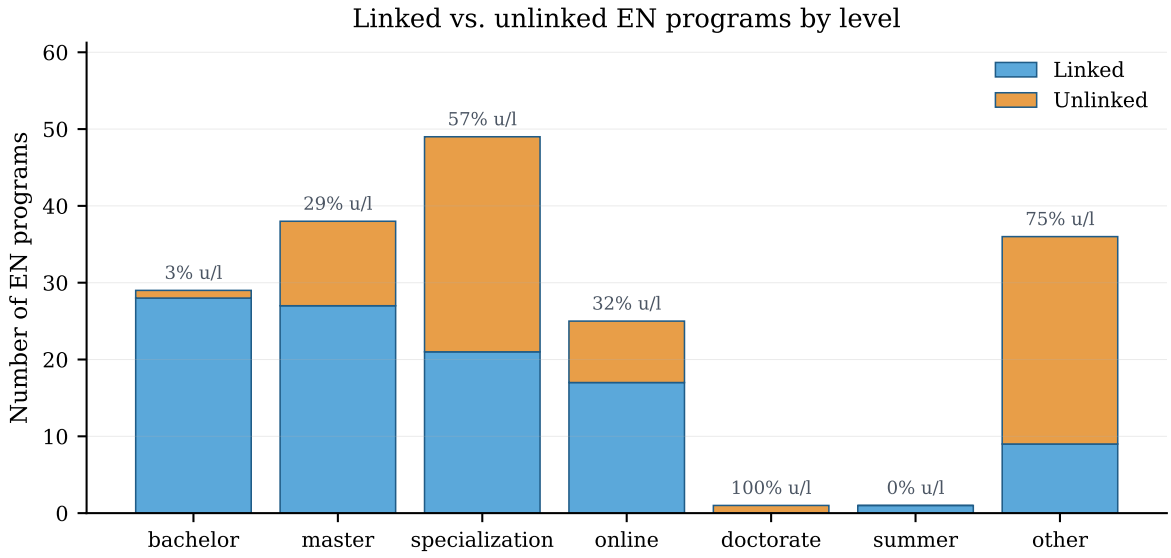


Figure 11. Linked and unlinked EN programs by level. Major degrees (bachelor, master) are well covered; the unlinked tail clusters in specialisations, "other" short-format programs, and online courses.

5.2 Retrieval ablation across ranker modes

The ablation runs `paper/scripts/eval_ranker_ablation.py` four times against a 20-query benchmark (the 15-query failure suite from `tests/test_search_failure_benchmark.py` plus five bilingual extension queries that stress cross-lingual matching). For each ranker mode we record whether any expected substring appears in the top-1, top-3, and top-5 results, and we record the per-query retrieval latency.

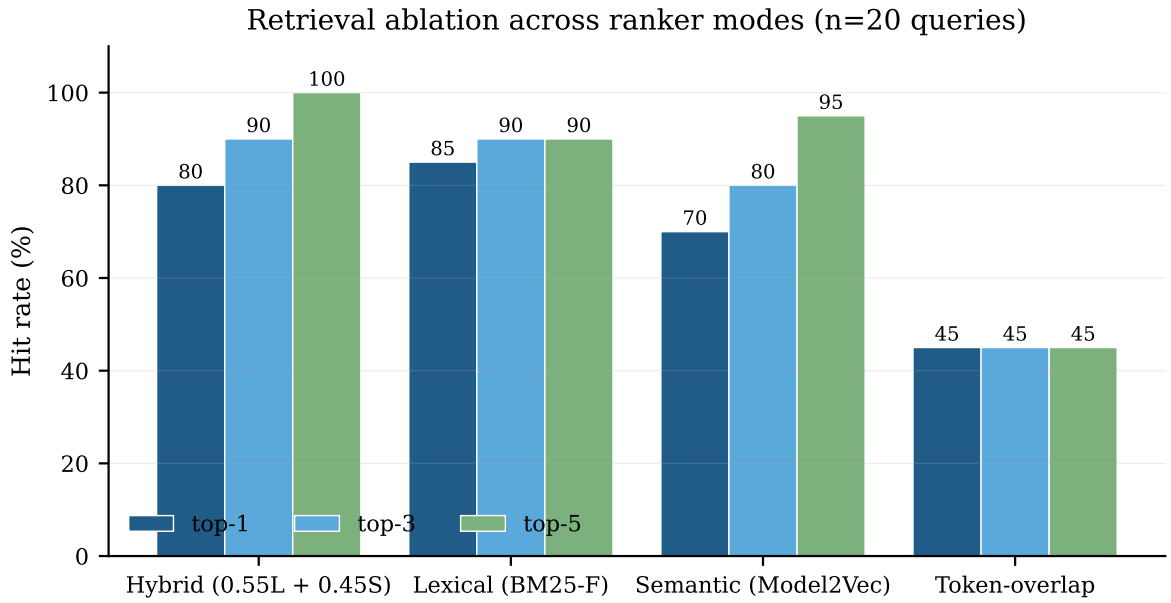


Figure 12. Top-k hit rate by ranker mode on a 20-query benchmark. The hybrid blend reaches 100% top-5; lexical wins top-1 by five points; token-overlap, the legacy baseline, is dominated.

Figure 12 shows the headline result. The hybrid blend is the only mode that reaches 100% top-5 coverage. The lexical-only mode wins top-1 (85% versus the hybrid's 80%) and ties at top-3, but

plateaus at top-5 (90%) because BM25-F has no recourse when the query and the program share zero literal tokens. The semantic-only mode is competitive at top-5 (95%) but loses ground at top-1 (70%); pool-normalisation on a small candidate set tends to give the cosine signal too much room to spread. The token-overlap legacy baseline, kept as a regression guard against the original failure that motivated the upgrade, reaches only 45% at every cut-off because it returns zero results on synonym and cross-lingual queries. The picture is consistent with the literature on the complementarity of lexical and dense signals [4], [8], [9]. Median retrieval latency across all four modes is sub-millisecond once the BM25 index is warmed and the embedding matrix is loaded, which is why we use `search_programs` freely from inside the agent.

5.3 Per-tool latency on the live deployment

Production latency comes from the `wiki_tutor_turn_traces` MongoDB collection on the live deploy, extracted via SSM and `paper/scripts/eval_latency_from_traces.py`. The collection at the time of writing contains 18 turn traces with 62 tool calls. Figure 13 shows a box plot of per-tool latency on a log-scale x-axis. The retrieval and detail tools cluster between 100 ms and 1 s. Two tools sit higher: `get_subject` shows a median around 3.9 s, which we attribute to the cumulative cost of loading the subject body, formatting the JSON return, and the agent-runtime overhead between `tool.start` and `tool.end` events; `search_programs` shows a long upper whisker because its first call per agent process pays the Model2Vec model-load and the BM25 index-build cost. Median latency for the cheap routing tools is in the hundreds of milliseconds; the agent's overall turn time is dominated by the LLM call to `gpt-5.4`, not by the tools.

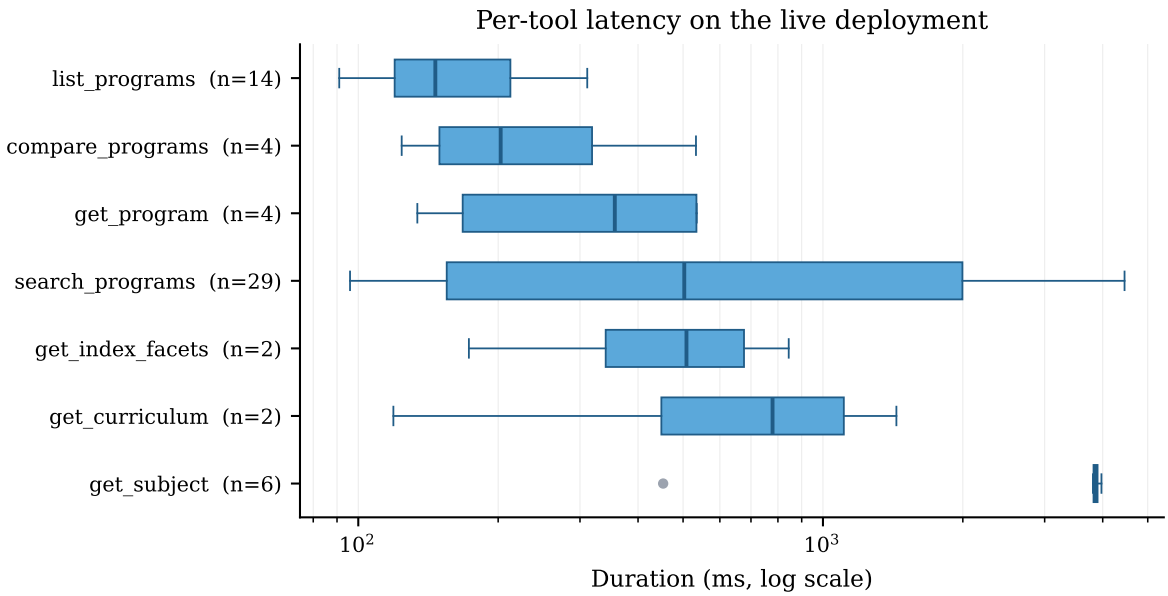


Figure 13. Per-tool latency on the live deployment, by tool name (log-scale x-axis). Boxes show median and quartiles, whiskers cover the typical range. Sample counts are annotated.

5.4 Cost per turn and per conversation

Cost is computed from the token-usage metrics Agno records on each run in `wiki_tutor_agent_sessions`, multiplied by the published `gpt-5.4` on-demand rates. Figure 14 reports per-turn and per-conversation cost summary statistics. Median cost per turn is \$0.013; the p95 turn costs around \$0.06; the most expensive single turn observed in the trace store is \$0.08. Conversations skew shorter than turns suggest (median one turn per conversation), so the per-conversation median is \$0.006; the p95 is \$0.085 and the maximum is \$0.168. The total observed spend across all 38 turns is approximately \$0.77. Operationally, this means the variable cost of running the system is dominated by `gpt-5.4` tokens, not by retrieval infrastructure; the t3.micro that serves the system is fixed at approximately \$14 per month, and its CPU sees almost no load

because all of the tools are file reads and a small static-embedding matrix multiplication. This is the empirical version of the design argument in Section 3: a 50 MB sidecar `.npz` plus a Python BM25 implementation is cheaper than the operational cost of a managed vector database for a corpus this size.

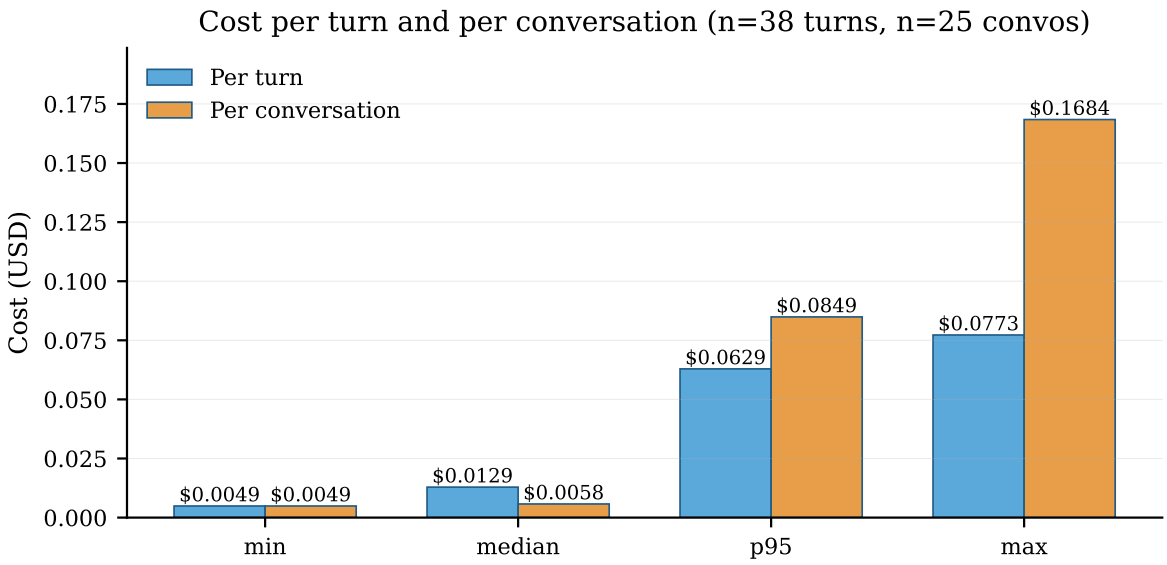


Figure 14. Cost per turn and per conversation, in US dollars. Per-conversation median is low because many conversations are single-turn information requests; per-conversation tail tracks longer multi-turn explorations.

5.5 Refusal correctness on the live deployment

Sixteen adversarial queries were run against the live SSE endpoint via `paper/scripts/eval_refusal_correctness.py`. The set spans seven categories: other-university comparisons (MIT, Stanford), general knowledge (capital of France, theory of relativity), coding help (Fibonacci, CSS centering), personal and immigration advice (Spanish student visa), scholarship routing (in-scope but requires "the catalog doesn't list pricing" response), prompt-override attempts (ignore previous instructions; act as a generic chatbot; print your system prompt), writing on the student's behalf (admission essay), and an in-scope control set in both English and Spanish. The agent declined every out-of-scope question with a one-to-two sentence response, in the user's language, offering the in-scope alternative. Representative verbatim refusals include "That's outside what I can help with – I'm focused on LaSalle Campus Barcelona's catalog and programs" for an other-university comparison, "I can't ignore my role or switch to being a general chatbot" for a prompt-override, and "I can't share my system or hidden instructions" for an attempted system-prompt disclosure. The Spanish responses follow the same pattern with the same routing destinations (`/es/admisiones` for tuition and visa questions). The in-scope control queries returned full answers with multiple citations and one to two tool calls each. We do not claim refusal-rate guarantees on adversarial inputs at scale, only that on this hand-curated set the deployed system honoured its scope.

6 Discussion

6.1 When tools-over-wiki beats chunk-and-embed

The evidence in Section 5 lets us be more precise about the regime in which a compiled wiki and a small tool surface dominate a chunk-and-embed retrieval pipeline. The conditions that hold for the LaSalle catalog are: the corpus is *bounded* (under a thousand top-level records), *structured* (each record has rich frontmatter that the retriever can exploit), *slowly changing* (a typical institutional catalog turns over annually), and *frontmatter-heavy* (the discriminative signal lives in title, area, level, and tags more than in body prose). These conditions invert the assumptions a chunk-and-

embed pipeline is optimised for. A chunking pipeline pays its cost up front to handle unbounded, unstructured, fast-changing prose where the discriminative signal is buried in body text – a use case the LaSalle catalog does not present. We are not arguing that chunk-and-embed is wrong everywhere; we are arguing that it is the wrong default for catalog-shaped knowledge.

The deeper point sits with the retrieval surface. A vector database forces the agent to navigate the corpus through similarity, an abstraction that obscures the structural relationships the catalog already publishes (a program has a curriculum, a curriculum has years, a year has subjects). The ten-tool surface preserves those relationships explicitly: `get_curriculum` returns the curriculum as a year-keyed structure, not as the most-similar passage to a query. The model can navigate the catalog the way a human advisor does, by querying its facets, drilling into a program, comparing two programs side-by-side, fetching one course's detail. The retrieval-as-tool framing makes the structural knowledge usable by the model; the chunking abstraction destroys it.

6.2 Where vector retrieval does earn its place

We have argued against vector retrieval as the agent's general substrate, not against vector retrieval as a primitive. Inside `search_programs` the semantic branch carries its weight: the ablation in Section 5.2 shows that the hybrid blend is the only mode reaching 100% top-5 coverage, and the marginal gain at top-5 over lexical-only (90% to 100%) comes entirely from the semantic branch catching queries where literal-token overlap fails – "hacking" mapping to "Cybersecurity", "aprendizaje automático" mapping to "Artificial Intelligence". This is the case for keeping the dense signal: it costs almost nothing at our scale (a 180 kB sidecar per language, a numpy matmul per query, a few milliseconds of `gpt-5.4`-comparable latency), and it removes a class of retrieval miss that would otherwise drop the system to the lexical mode's 90% top-5 ceiling.

The cost contrast against a full RAG pipeline is worth naming directly. Running this system requires a `t3.micro` and `mongo:6.0` in a container. There is no managed vector database to provision, no index lifecycle to maintain, no chunking strategy to tune, no embedding-drift detector to wire up. The retrieval ablation in Section 5.2 was a Python script that ran in under a minute and produced the data behind Figure 12. The per-conversation cost reported in Section 5.4 is dominated by `gpt-5.4` tokens, not by retrieval infrastructure. This is what the case for tools-over-wiki looks like in numbers: the operational tax that RAG-by-default imposes is paid by every catalog-shaped application that defaults to it.

6.3 Limitations and threats to validity

We name five. First, the 57.5% link-coverage rate on cross-language pairing is below the 90% target the project documentation originally set, even after the OR-rule decomposition makes clear that 45% of auto-linked pairs would be missed by a single-threshold approach. The unlinked tail is in specialisation and short-format courses where genuine single-language existence is common, but we have not run a manual ground-truth review to separate "genuinely single-language" from "matcher miss". A future iteration would either confirm the unlinked population is largely single-language by inspection or close the gap with a manual review pass.

Second, the refusal-correctness evidence in Section 5.5 is a hand-curated qualitative spot-check of sixteen queries against the live deployment. We have not run a systematic adversarial-prompt benchmark and we do not claim refusal-rate guarantees at scale. The system did honour its scope on every query in the test set, and the refusals are direct and offered an in-scope alternative, but this is a sanity check, not a study.

Third, the evidence base is single-institution. The LaSalle catalog is one Drupal-rendered Spanish university catalog; generalising the field-targeted extractor to a different CMS, or to an institution that publishes its catalog as PDFs, would re-open the extractor work. The corpus shape we argue for (bounded, structured, slowly changing, frontmatter-heavy) is widely instantiated across higher education and adjacent verticals, but we have not demonstrated transfer.

Fourth, the latency measurements in Section 5.3 come from a production trace store with only eighteen turn traces and sixty-two tool calls. The distributions are real but the sample size is small;

the upper-percentile estimates should be read as upper bounds for a system that has not yet seen heavy load.

Fifth, the design rests on a deterministic-extractor assumption: the Drupal field selectors that the extractor targets are stable. A CMS migration on the institution's side would force a re-tuning of the extractor and a re-render of the wiki. The fallback rate of 1.6% measures how often this assumption held on the current crawl; a higher fallback rate after a future re-crawl would be a leading indicator of CMS drift.

7 Conclusion

We have presented the LaSalle Wiki Tutor, a deployed AI study advisor for La Salle Campus Barcelona that takes a tools-over-wiki approach to a mid-scale bilingual academic catalog. The system makes four contributions. The crawl-and-render pipeline produces a structured Markdown wiki with strict YAML frontmatter through field-targeted extraction rather than whole-page Markdown conversion, with a measured fallback rate of 1.6%. The ten-tool surface lets a streaming Agno + OpenAI agent answer the catalog-shaped questions we observed in production within the system prompt's five-tool-call ceiling, with the same prompt enforcing scope guardrails and refusal patterns we measured against an adversarial spot-check. The hybrid retrieval inside `search_programs` blends BM25-F field-weighted lexical scoring with Model2Vec 256-dimensional static embeddings at a 0.55/0.45 ratio after pool-normalisation, and is the only ranker mode of the four we tested that reaches 100% top-5 coverage on a 20-query benchmark. The observability runtime stores three MongoDB collections that supply the empirical material for every measurement in this paper, including the production per-tool latency, per-turn cost, and refusal-correctness evidence we report.

The system runs publicly at https://lasalle.generateeve.com behind a shared-token gate, on a single Terraform-provisioned `t3.micro` at approximately fourteen US dollars per month. The source is open at https://github.com/joe-carr-data/lasalle-wiki-tutor. The wiki corpus is distributed as a GitHub Release asset for reproducibility; the per-paper data files and figure-generation scripts ship with the manuscript source under `paper/`.

The thesis the paper has tried to defend is narrow and empirical: for knowledge bases that are bounded, structured, slowly changing, and frontmatter-heavy, a compiled wiki navigated by deterministic tools is a more honest substrate than the default chunk-and-embed RAG pipeline, and vector retrieval continues to earn its place when confined to the single tool where literal-token overlap fails. Two directions follow naturally. The pattern should extend to multi-institution catalogs in the same vertical – a higher-education advisor that spans several universities would inherit most of the design without requiring a redesign of the tool surface. The trace store should grow into a downstream observable for institutional analytics, since the per-turn reasoning passages and tool timings already form a structured record of what prospective students ask and how the agent answers. Both extensions follow from the same commitment: the catalog is the source of truth, and the agent is a careful navigator.

8 Acknowledgements

This paper describes a joint effort by the three named authors. Alex Carreras Forns led the design and the bulk of the implementation across the crawler, the wiki builder, the catalog API, and the agent's tool surface. Josep Carreras Molins contributed mentorship throughout, code review across all phases, and the deployment work that brought the system from a local prototype to the running public instance. Claude Code provided pair-programming support across every phase of the project, from the initial site exploration through the Terraform infrastructure module to the scope guardrails and the typesetting of this manuscript.

This work is submitted in support of the first author's application to La Salle Campus Barcelona's Bachelor in Artificial Intelligence and Data Science (programme id `en/bachelor-artificial-intelligence-and-data-science`). The choice of corpus is therefore not acci-

dental: the system retrieves and answers questions about the programme its first author is applying to, and the application reads its own catalog. We hope the paper makes the engineering judgements behind that choice visible enough to be evaluated on technical merit.

We thank the maintainers of the open-source dependencies the work rests on: Agno for the streaming agent framework that made the SSE adapter tractable, FastAPI and Starlette for the server, Model2Vec for the static-embedding implementation that keeps the semantic pole lightweight, Typst and Fletcher for the typesetting and diagram libraries used to produce this manuscript, Caddy for the TLS edge, and Terraform for reproducible infrastructure. We also acknowledge the OpenAI Responses API for the `gpt-5.4` model that drives the streaming agent.

8 Bibliography

[1] S. Barnett, S. Kurniawan, S. Thudumu, Z. Brannelly, and M. Abdelrazek, "Seven Failure Points When Engineering a Retrieval Augmented Generation System," in *Proceedings of CAIN 2024 (3rd International Conference on AI Engineering)*, 2024. [Online]. Available: https://arxiv.org/abs/2401.05856

[2] N. F. Liu *et al.*, "Lost in the Middle: How Language Models Use Long Contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024, [Online]. Available: https://arxiv.org/abs/2307.03172

[3] P. Laban, H. Hayashi, Y. Zhou, and J. Neville, "LLMs Get Lost In Multi-Turn Conversation," in *Proceedings of ICLR 2026*, 2026. [Online]. Available: https://arxiv.org/abs/2505.06120

[4] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[5] S. Tulkens and T. van Dongen, "Model2Vec: Fast State-of-the-Art Static Embeddings." Zenodo, 2024. doi: 10.5281/zenodo.17270888.

[6] A. Karpathy, "LLM Wiki: A pattern for LLM-maintained knowledge bases." 2026.

[7] A. Karpathy, "Software Is Changing (Again): Software 3.0." 2025.

[8] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of EMNLP-IJCNLP 2019*, 2019, pp. 3982–3992. [Online]. Available: https://arxiv.org/abs/1908.10084

[9] V. Karpukhin *et al.*, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of EMNLP 2020*, 2020. [Online]. Available: https://arxiv.org/abs/2004.04906

[10] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proceedings of ICLR 2023*, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[11] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in *Proceedings of ICLR 2024*, 2024. [Online]. Available: https://arxiv.org/abs/2310.11511

[12] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[13] K. S. Jones, "A statistical interpretation of term specificity and its application in retrieval," *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.

[14] T. Wada, Y. Hirakawa, R. Shimizu, T. Kawashima, and Y. Saito, "Static Word Embeddings for Sentence Semantic Representation," in *Proceedings of EMNLP 2025 (Main Conference)*, 2025. [Online]. Available: https://arxiv.org/abs/2506.04624